

Практическое занятие №2: Основы ООП

Цель данного занятия — изучить основные принципы объектно-ориентированного программирования (ООП) на языке Java и научиться применять их на практике. Для этого нужно освоить создание простых классов и объектов и научиться писать код, в котором они взаимодействуют между собой.

Для этого нужно освоить:

- - создание классов и объектов;
- - работу с полями и методами класса;
- - принципы инкапсуляции и модификаторы доступа;
- - использование конструкторов;

Объектно-ориентированное программирование — это парадигма, основанная на концепции объектов, которые содержат данные (поля) и методы для работы с этими данными.

В качестве примера будем решать задачу расчета площади и периметра геометрических фигур. Начнем с окружности.

Версия 1

Сначала напишем код в процедурном стиле. У нас будут статические методы для расчета площади и периметра, а также массив, где будут храниться радиусы окружностей.

```
1 // Main.java
2
3 public class Main {
4     public static double circleArea(double r) {
5         // Площадь круга
6         return Math.PI * r * r;
7     }
8
9     public static double circlePerimeter(double r) {
10        // Периметр круга
11        return 2.0 * Math.PI * r;
12    }
13
14    public static void main(String[] args) {
15
16        // Массив с радиусами
17        double[] radius = { 3.0, 2.5, 5.0, 1.0, 20.0 };
18
19        // Цикл для расчета площадей
20        for (double r : radius) {
21            double area = circleArea(r);
22            double perimeter = circlePerimeter(r);
23            System.out.println("Радиус окружности: " + r);
```

```
24         System.out.println("Площадь: " + area);
25         System.out.println("Периметр: " + perimeter);
26     }
27 }
28 }
```

Пока мы имеем дело с окружностью, у которой для расчета параметров нужен только радиус, больших проблем нет. Перейдем к прямоугольнику. У него есть ширина и высота. Для их хранения можно создать 2 массива, в одном хранить ширины прямоугольников, в другом — высоты.

Версия 2

Напишем метод расчета площади и периметра прямоугольника.

```
1  // Main.java
2
3  public class Main {
4      public static double rectangleArea(double w, double h) {
5          // Площадь прямоугольника
6          return w * h;
7      }
8
9      public static double rectanglePerimeter(double w, double h) {
10         // Периметр прямоугольника
11         return 2.0 * (w + h);
12     }
13
14     public static void main(String[] args) {
15
16         // Массив с ширинами
17         double[] width = {3.0, 2.5, 5.0};
18         // Массив с высотами
19         double[] height = {2.0, 5.6, 1.0};
20
21         // Цикл для расчета площадей
22         for (int i = 0; i < width.length; i++) {
23             double area = rectangleArea(width[i], height[i]);
24             double perimeter =
25                 rectanglePerimeter(width[i], height[i]);
26             System.out.println("Стороны: " + w + " и " + h);
27             System.out.println("Площадь: " + area);
28             System.out.println("Периметр: " + perimeter);
29         }
30     }
31 }
```

Появились потенциальные проблемы. Массивы независимы друг от друга, можно случайно забыть добавить, например, высоту для одного из прямоугольников. Данные не связаны между собой, хотя относятся к одному объекту — прямоугольнику. Как вариант решения проблемы, мы можем попробовать для хранения сторон прямоугольника использовать двумерный массив, но код получится сложнее, да и это разовое решение, может получиться так, что данные будут разных типов, и мы не сможем хранить их в одном массиве.

Версия 3

Для решения проблемы нам нужно как-то объединить в одной сущности и ширину и высоту прямоугольника. Для этого как раз подходит тип данных — класс. Структуру класса мы можем задать сами. Создадим класс `Rectangle`, в котором будут одновременно храниться 2 переменных (или как их еще называют, 2 поля) — ширина и высота. Создадим конструктор класса для создания объектов. Сами объекты будут хранить в массиве прямоугольников. Соответственно, в методы для расчета площади и периметра мы можем передавать сразу целый объект класса `Rectangle`, а из него получать размеры сторон для расчета.

```
1 // Rectangle.java
2
3 public class Rectangle {
4     double width;
5     double height;
6     Rectangle (double w, double h) {
7         width = w;
8         height = h;
9     }
10 }
```

```
1 // Main.java
2
3 public class Main {
4     public static double
5         rectangleArea(Rectangle rectangle) {
6         // Площадь прямоугольника
7         return rectangle.w * rectangle.h;
8     }
9
10    public static double
11        rectanglePerimeter(Rectangle rectangle) {
12        // Периметр прямоугольника
13        return 2.0 * (rectangle.w + rectangle.h);
14    }
15
16    public static void main(String[] args) {
17
18        // Массив прямоугольников
19        Rectangle[] rectangles = {
20            new Rectangle(3.0, 2.0),
```

```

21         new Rectangle(2.5, 5.6),
22         new Rectangle(5.0, 1.0)
23     };
24
25     // Цикл для расчета площадей
26     for (Rectangle r : rectangles) {
27         double area = rectangleArea(r);
28         double perimeter = rectanglePerimeter(r);
29         System.out.println("Стороны: " + r.w + " и " + r.h);
30         System.out.println("Площадь: " + area);
31         System.out.println("Периметр: " + perimeter);
32     }
33 }
34 }

```

Код стал более структурированным и понятным, но если вернуть возможность считать еще и площадь и периметр круга, а потом еще потенциально и другие фигуры, то получится, что наш класс Main оказывается перегруженным разнообразными методами для расчета разных фигур, что приводит к усложнению понимания кода. Более правильный вариант — это переместить методы, рассчитывающие параметры фигуры в код ее класса.

Версия 4

Доработаем код класса Rectangle путем переноса в него методов для расчета площади и периметра.

```

1  // Rectangle.java
2
3  public class Rectangle {
4      double width;
5      double height;
6
7      Rectangle (double w, double h) {
8          width = w;
9          height = h;
10     }
11     public double getArea() {
12         // Площадь
13         return width * height;
14     }
15     public double getPerimeter() {
16         // Периметр
17         return 2.0 * (width + height);
18     }
19 }

```

```

1  // Main.java
2

```

```

3 public class Main {
4
5     public static void main(String[] args) {
6
7         // Массив прямоугольников
8         Rectangle[] rectangles = {
9             new Rectangle(3.0, 2.0),
10            new Rectangle(2.5, 5.6),
11            new Rectangle(5.0, 1.0)
12        };
13
14        // Цикл для расчета площадей
15        for (Rectangle r : rectangles) {
16            System.out.println("Стороны прямоугольника: " +
17                r.w + " и " + r.h);
18            System.out.println("Площадь: " + r.getArea());
19            System.out.println("Периметр: " + r.getPerimeter());
20        }
21    }
22 }

```

Обратите внимание, что код методов стал проще и короче. Они теперь не статические, а методы класса, и расположены в самом классе, поэтому методу более не нужно получать объект прямоугольника извне, поля объекта доступны и так.

Версия 5

Вернем возможность считать параметры окружности. Создадим класс Circle. Помимо этого, добавим классам фигур переопределенный метод `toString()`, чтобы было проще выводить данные об объекте. Поля можно сделать закрытыми (`private`), чтобы не разрешать доступ к ним вне кода класса — это больше не нужно и такой вариант более безопасный.

```

1 // Rectangle.java
2
3 public class Rectangle {
4     private double width;
5     private double height;
6
7     Rectangle (double w, double h) {
8         width = w;
9         height = h;
10    }
11    public double getArea() {
12        // Площадь
13        return width * height;
14    }
15    public double getPerimeter() {
16        // Периметр

```

```
17     return 2.0 * (width + height);
18 }
19 @Override public String toString() {
20     return "Прямоугольник со сторонами " +
21         width + " и " + height;
22 }
23 }
```

```
1 // Circle.java
2
3 public class Circle {
4     private double radius;
5
6     Rectangle (double r) {
7         radius = r;
8     }
9     public double getArea() {
10         // Площадь
11         return Math.PI * radius * radius;
12     }
13     public double getPerimeter() {
14         // Периметр
15         return 2.0 * Math.PI * radius;
16     }
17     @Override public String toString() {
18         return "Окружность с радиусом " + radius;
19     }
20 }
```

```
1 // Main.java
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         // Массив прямоугольников
8         Rectangle[] rectangles = {
9             new Rectangle(3.0, 2.0),
10             new Rectangle(2.5, 5.6),
11             new Rectangle(5.0, 1.0)
12         };
13
14         Circle[] circles = {
15             new Circle(3.0),
16             new Circle(10.0)
17         }
```

```

18
19     // Циклы для расчета параметров
20     for (Rectangle r : rectangles) {
21         System.out.println(r);
22         System.out.println("Площадь: " + r.getArea());
23         System.out.println("Периметр: " + r.getPerimeter());
24     }
25     for (Circle c : circles) {
26         System.out.println(c);
27         System.out.println("Площадь: " + c.getArea());
28         System.out.println("Периметр: " + c.getPerimeter());
29     }
30 }
1 }

```

Код, который выводит площадь и периметр — дублируется. Попробуем избавиться от дублирования кода.

Версия 6

Введем общий базовый класс Shape, в который поместим одинаковые для всех фигур действия. Класс будет абстрактным, так как при выводе описания нам нужно обращаться к методам расчета площади и периметра, а для произвольной фигуры мы не можем их рассчитать. Классы Rectangle и Circle являются наследниками класса Shape, поэтому метод printDescription будет унаследован от базового класса, а методы для расчета параметров будут свои в каждом классе. Метод toString тоже для каждого класса свой, но его можно не делать в классе Shape абстрактным, так как он определен еще в классе Object, поэтому он есть и в классе Shape, дополнительно объявлять его тут не нужно.

```

1  // Shape.java
2
3  abstract public class Shape {
4      // Любая фигура должна уметь печатать информацию о себе
5      public String printDescription() {
6          System.out.println(toString());
7          System.out.println("Площадь: " + getArea());
8          System.out.println("Периметр: " + getPerimeter());
9      }
10     // Любая фигура должна уметь вычислять свою площадь
11     abstract double getArea();
12     // Любая фигура должна уметь вычислять свой периметр
13     abstract double getPerimeter();
14 }

```

```

1  // Rectangle.java
2
3  public class Rectangle extends Shape {

```

```
4 private double width;
5 private double height;
6
7 Rectangle (double w, double h) {
8     width = w;
9     height = h;
10 }
11 @Override public double getArea() {
12     // Площадь
13     return width * height;
14 }
15 @Override public double getPerimeter() {
16     // Периметр
17     return 2.0 * (width + height);
18 }
19 @Override public String toString() {
20     return "Прямоугольник со сторонами " +
21         width + " и " + height;
22 }
23 }
```

```
1 // Circle.java
2
3 public class Circle extends Shape {
4     private double radius;
5
6     Rectangle (double r) {
7         radius = r;
8     }
9     public double getArea() {
10         // Площадь
11         return Math.PI * radius * radius;
12     }
13     public double getPerimeter() {
14         // Периметр
15         return 2.0 * Math.PI * radius;
16     }
17     @Override public String toString() {
18         return "Окружность с радиусом " + radius;
19     }
20 }
```

```
1 // Main.java
2
3 public class Main {
4
```

```

5    public static void main(String[] args) {
6
7        // Массив прямоугольников
8        Rectangle[] rectangles = {
9            new Rectangle(3.0, 2.0),
10           new Rectangle(2.5, 5.6),
11           new Rectangle(5.0, 1.0)
12        };
13
14        Circle[] circles = {
15            new Circle(3.0),
16            new Circle(10.0)
17        }
18
19        // Циклы для расчета параметров
20        for (Rectangle r : rectangles) {
21            r.printDescription();
22        }
23        for (Circle c : circles) {
24            c.printDescription();
25        }
26    }
27 }

```

Версия 7

Осталось только унифицировать обработку фигур в цикле. Так как и окружности, и прямоугольники являются потомками класса Shape, можно создать массив объектов класса Shape, в цикле у каждого будем вызывать метод printDescription(), внутри которого будут вызываться методы toString, getArea и getPerimeter, причем вызываться они будут у объектов в соответствии с их реальным классом. Приведем код метода Main, остальные классы остались без изменений. При этом можно добавить еще класс, соответствующий треугольнику. При добавлении нового класса нужно только задать ему поля, характеризующие размеры фигуры, добавить методы расчета площади и периметра, а также метод вывода типа фигуры с размерами. Весь остальной код трогать не нужно. Программа стала расширяемой за счет полиморфизма.

```

1    // Triangle.java
2
3    public class Triangle extends Shape {
4        private double a, b, c; // стороны
5
6        Rectangle (double a, double b, double c) {
7            this.a = a;
8            this.b = b;
9            this.c = c;
10       }

```

```

11  @Override public double getArea() {
12      // Площадь
13      double p = getPerimeter();
14      return Math.sqrt((p * (p - a) * (p - b) * (p - c)));
15  }
16  @Override public double getPerimeter() {
17      // Периметр
18      return a + b + c;
19  }
20  @Override public String toString() {
21      return "Треугольник со сторонами "+a+", "+b+", "+c;
22  }
23  }

```

```

1  // Main.java
2
3  public class Main {
4
5      public static void main(String[] args) {
6
7          // Массив фигур
8          Shape[] shapes = {
9              new Rectangle(3.0, 2.0),
10             new Triangle(3.0, 4.0, 5.0),
11             new Circle(10.0),
12             new Triangle(3.5, 3.5, 5.0),
13             new Rectangle(2.5, 5.6),
14             new Circle(3.0),
15             new Rectangle(5.0, 1.0)
16             new Triangle(3.5, 3.5, 5.0),
17         };
18
19         // Цикл для расчета параметров
20         for (Shape s : shapes) {
21             s.printDescription();
22         }
23     }
24 }

```

Для закрепления материала попробуйте самостоятельно:

1. Создать класс Animal с полями name и age и методом makeSound.
2. Наследовать от него классы Dog и Cat, переопределив метод makeSound.
3. Использовать полиморфизм для вызова методов разных классов через массив Animal[].
4. Добавить статическое поле count для подсчёта созданных объектов.

Лабораторная работа №2

На основе базового класса `Pokemon` написать свои классы для заданных видов покемонов. Каждый вид покемона должен иметь один или два типа и стандартные базовые характеристики:

- очки здоровья (HP)
- атака (attack)
- защита (defense)
- специальная атака (special attack)
- специальная защита (special defense)
- скорость (speed)

Классы покемонов должны наследоваться в соответствии с цепочкой эволюции покемонов. На основе базовых классов `PhysicalMove`, `SpecialMove` и `StatusMove` реализовать свои классы для заданных видов атак. Все разработанные классы, не имеющие наследников, должны быть реализованы таким образом, чтобы от них нельзя было наследоваться.

Атака должна иметь стандартные тип, силу (power) и точность (accuracy). Должны быть реализованы стандартные эффекты атаки. Назначить каждому виду покемонов атаки в соответствии с вариантом. Уровень покемона выбирается минимально необходимым для всех реализованных атак.

Используя класс симуляции боя `Battle`, создать 2 команды покемонов (каждый покемон должен иметь имя) и запустить бой.

Пример выполнения работы

Цель работы: на простом примере разобраться с основными концепциями ООП и научиться использовать их в программах.

В этой части мы создадим модели сражения покемонов. При этом базовые классы для покемона и атаки уже написаны и расположены в jar-архиве `Pokemon.jar`. Нам нужно только реализовать наследников этих классов в соответствии с заданием.

Этапы работы:

1. Ознакомьтесь с документацией на сайте se.ifmo.ru по классам, входящим в jar-архив, обращая особое внимание на классы `Battle`, `Pokemon` и `Move`. Посмотрите какие есть методы в этих классах, изучите описания этих методов.
2. Скачайте файл `Pokemon.jar`. Его необходимо будет использовать как для компиляции, так и для запуска программы. Распаковывать его не надо!
3. Напишем минимально работающую программу и посмотрим как она работает.

```
1 // Main.java
2
3 import ru.ifmo.se.pokemon.*;
4
```

```

5 public class Main {
6     public static void main(String[] args) {
7         Battle b = new Battle();
8         Pokemon p1 = new Pokemon("Чужой", 1);
9         Pokemon p2 = new Pokemon("Хищник", 1);
10        b.addAlly(p1);
11        b.addFoe(p2);
12        b.go();
13    }
14 }

```

Для того, чтобы компилятор мог узнать о классах, входящих в архив Pokemon.jar, о месте их нахождения нужно сообщить компилятору.

```
javac -cp ./Pokemon.jar:. Main.java
```

Опция -cp задает список путей, где компилятор будет искать классы. В данном случае это архив Pokemon.jar и текущий каталог (обозначающийся точкой). В конце указываем какие классы нужно откомпилировать — Main.java.

После компиляции запустим получившийся класс Main:

```
java -cp ./Pokemon.jar:. Main
```

Должно запуститься сражение двух покемонов — Чужого и Хищника, но применять они будут только стандартные базовые атаки, потому что ничему другому они не обучены.

Теперь попробуем реализовать свои собственные классы для покемона и атаки — допустим, что это будет ItMonster с собственной атакой NonClassicMove. Разместим класс покемона в пакете pokemon, а класс атаки — в пакете move.

Класс атаки NonClassicMove должен наследоваться от базового класса атаки PhysicalMove. В конструкторе нужно будет задать тип атаки, ее силу и точность. Вызываем конструктор суперкласса с нужными нам параметрами, типом, мощностью, точностью, приоритетом и количеством ударов. Зададим для нашей атаки тип — психическая, мощность — 100, точность — 1.0, и установим количество ударов — 3 за один ход.

Кроме этого в атаке нужно будет переопределить метод describe, чтобы выводилось нужное сообщение. Если действие атаки отличается от стандартного, например, покемон не промахивается, либо атакующий покемон также получает повреждение, то в классе атаки нужно дополнительно переопределить соответствующие методы.

При реализации атак, которые меняют статус покемона, придется разобраться с классом Effect. Он позволяет на один или несколько ходов изменить состояние покемона или модификатор его базовых характеристик. Будем считать, что данная атака накладывает на соперника эффект растерянности с вероятностью 50%. Поэтому переопределим метод applyOppEffects.

```

1 // NonClassicMove.java
2 package move;
3
4 import ru.ifmo.se.pokemon.*;
5 import pokemon.*;

```

```

6
7 public class NonClassicMove extends PhysicalMove {
8
9     public NonClassicMove() {
10         super(Type.PSYCHIC, 100, 1.0, 0, 3);
11     }
12
13     @Override public void applyOppEffects(Pokemon p) {
14         if (Math.random() < 0.5) {
15             p.confuse();
16         }
17     }
18
19     @Override public String describe() {
20         return "применяет неклассическую психическую атаку";
21     }
22 }

```

Класс покемона ItMonster должен наследоваться от базового класса Pokemon. В конструкторе нужно будет задать тип покемона и его базовые характеристики. Вызываем конструктор суперкласса с именем и уровнем, устанавливаем тип и характеристики покемона, и добавляем атаку NonClassicMove, чтобы покемон мог ей пользоваться.

```

1 // ItMonster.java
2 package pokemon;
3
4 import ru.ifmo.se.pokemon.*;
5 import move.*;
6
7 public class ItMonster extends Pokemon {
8
9     public ItMonster(String name, int level) {
10         super(name, level);
11         setType(Type.PSYCHIC);
12         setStats(50, 50, 50, 50, 50, 50);
13         this.addMove(new NonClassicMove());
14     }
15 }

```

Теперь переместим Чужого в команду к Хищнику, а соперником у них будет наш новый ItMonster по имени Монстрик. Класс Main поместим в пакет lab2.

```

1 // Main.java
2 package lab2;
3 import ru.ifmo.se.pokemon.*;
4 import pokemon.*;
5 import move.*;
6

```

```

7 public class Main {
8     public static void main(String[] args) {
9         Battle b = new Battle();
10        Pokemon p1 = new Pokemon("Чужой", 1);
11        Pokemon p2 = new Pokemon("Хищник", 1);
12        b.addAlly(new ItMonster("Монстрик", 5));
13        b.addFoe(p1);
14        b.addFoe(p2);
15        b.go();
16    }
17 }

```

Так как классов стало больше и они распределены по пакетам, то используем опцию `-d`, чтобы компилятор распределил получившиеся классы в соответствующий пакет.

```
javac -cp ./Pokemon.jar:. -d . *.java
```

Опция `-d` задает корневой каталог, куда компилятор будет складывать классы в соответствии со структурой пакета. В конце указываем какие классы нужно откомпилировать — все файлы с расширением `.java`.

После выполнения компиляции файлы классов будут размещены по каталогам в соответствии с их распределением по пакетам:

```

.
├── lab2
│   └── Main.class
├── move
│   └── NonClassicMove.class
└── pokemon
    └── ItMonster.class

```

Запустим получившийся класс `Main` с указанием его полного имени, включая имя пакета `l2`:

```
java -cp ./Pokemon.jar:. l2.Main
```

Должно запуститься сражение Монстрика, применяющего тройные неклассические атаки против Чужого и Хищника со стандартными базовыми атаками.

По такому же принципу реализуйте самостоятельно атаки и покемонов для своего варианта.

Информацию о покемонах, цепочках эволюции и атаках можно найти на сайтах

- <http://poke-universe.ru>
- <http://pokemondb.net>
- <http://veekun.com/dex/pokemon>

Содержание отчета по работе

1. Текст задания.
2. Диаграмма классов реализованной объектной модели.
3. Исходный код программы.

4. Результат работы программы.
5. Выводы по работе.

Вопросы к защите

1. Объектно-ориентированное программирование. Основные понятия: объекты, наследование, полиморфизм, инкапсуляция.
2. Понятие класса. Классы и объекты в Java.
3. Члены класса. Поля, методы, конструкторы. Модификаторы доступа.
4. Создание и инициализация объектов. Вызов методов.
5. Области видимости переменных.
6. Модификаторы `final` и `static`.
7. Пакеты, инструкция `import`.